

Faculty of Degree Engineering – 047

Department of Computer Engineering – 07

SEMESTER: 4

LAB MANUAL

Analysis And Design of Algorithms
BE04000241

Name: _____

Enrollment No: _____

Batch: _____

DEPARTMENT OF COMPUTER ENGINEERING

V.V.P. ENGINEERING COLLEGE



Faculty of Degree Engineering – 047

Department of Computer Engineering – 07

INDEX

SR. NO.	LIST OF EXPERIMENTS	DATE	SIGN	RE-Marks
1	Perform following sorting operation and measure the execution time for sufficient large input: Selection Sort, Bubble Sort, and Insertion Sort.			
2	Implement Linear search and Binary Search			
3	Perform following sorting operation and measure the execution time for sufficient large input: Merge Sort, Quick Sort			
4	Solve Given Problems using Greedy Strategy: Make a change problem, Activity Selection			
5	Solve Given Problems using Greedy Strategy: Problem, 0/1 Knapsack Problem, Job Scheduling Problem			
6	Solve Given Problems using Greedy Strategy: MST using Prim's Algorithm, MST using Krushkal's Algorithm			
7	Solve Given Problems Using Dynamic Programming: Make a change problem, 0/1 Knapsack Problem			
8	Solve Given Problems Using Dynamic Programming: Matrix Chain Multiplication			
9	Solve Given Problems Using Dynamic Programming: Longest Common Subsequence (LCS), Shortest Common Supper sequence (SCS)			
10	Solve following problems using Backtracking 4-Queen Problem, 8-Queen Problem			

Practical 7

Title: Implementation of Making Change Problem using Dynamic Programming (Matrix Method)

Objective: To implement the Making Change Problem using the Dynamic Programming approach (matrix/table method) and determine the minimum number of coins required to make a given amount

Theory: The Making Change Problem is an optimization problem where the objective is to find the minimum number of coins required to make a given amount from a set of available coin denominations. Unlike the Greedy approach, Dynamic Programming (DP) guarantees an optimal solution for all coin systems. In this method, a 2D matrix (table) is constructed where each entry represents the minimum number of coins required to make a certain amount using a limited set of coins. The solution builds incrementally by solving smaller subproblems and storing their results in a matrix to avoid recomputation.

Working Principle (Matrix Method):

- Rows represent coin denominations.
- Columns represent amounts from 0 to the required value.
- Each cell stores the minimum number of coins needed.
- The final answer is obtained from the last cell of the matrix.

Characteristics:

- Always produces an optimal solution.
- Uses extra memory (table).
- More reliable than Greedy approach.
- Suitable for all coin denomination systems.

Time Complexity:

- Time Complexity: O(n × amount)

Space Complexity: O(n × amount) Where n is the number of coin denomination.

Algorithm / Steps:

1. Start the algorithm.
2. Read number of coin denominations.
3. Read the coin values.
4. Read the total amount.
5. Create a 2D array dp[n+1][amount+1].
6. Initialize: o dp[i][0] = 0 (0 coins needed for amount 0) o dp[0][j] = ∞ (impossible without coins)
7. Fill the matrix using: o If coin value ≤ current amount dp[i][j] = min(dp[i- 1][j], 1 + dp[i][j - coin[i]]) o Else dp[i][j] = dp[i-1][j]
8. Display minimum coins required.

Code:

```
#include <stdio.h>
#include <limits.h>

int main() {
   int n, amount;

   printf("Enter number of coin types: ");
   scanf("%d", &n);

   int D[n];
   printf("Enter coin denominations:\n");
   for(int i = 0; i < n; i++) {
      scanf("%d", &D[i]);
   }

   printf("Enter amount: ");
   scanf("%d", &amount);

   int C[n][amount + 1];

   for(int i = 0; i < n; i++) {
      C[i][0] = 0;
   }

   for(int i = 0; i < n; i++) {
      for(int j = 1; j <= amount; j++) {

         if(i == 0) {
            if(j % D[i] == 0) {
               C[i][j] = j / D[i];
            } else {
               C[i][j] = INT_MAX;
            }
         }
         else if(j < D[i]) {
            C[i][j] = C[i-1][j];
         }
         else {
            int alt;
            if(C[i][j - D[i]] == INT_MAX)
               alt = INT_MAX;
            else
               alt = 1 + C[i][j - D[i]];

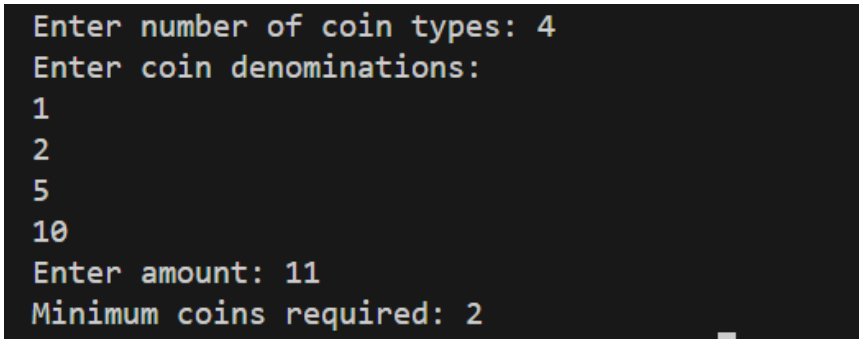
            if(C[i-1][j] < alt)
               C[i][j] = C[i-1][j];
            else
               C[i][j] = alt;
         }
      }
```

```
    }

    if(C[n-1][amount] == INT_MAX) {
        printf("Not Possible\n");
    } else {
        printf("Minimum coins required: %d\n", C[n-1][amount]);
    }

    return 0;
```

Output



Practical 7

Title: Implementation of 0/1 Knapsack using Dynamic Programming (Matrix Method)
Objective: To implement the 0/1 Knapsack using the Dynamic Programming approach (matrix/table method) and determine the Maximum profit and items selected.

Theory: The 0/1 Knapsack Problem is a classic optimization problem in Dynamic Programming. The 0/1 Knapsack Problem is a classic optimization problem in Dynamic Programming. Given a set of items, each with a weight and a value, and a knapsack with limited capacity, the goal is to maximize the total value without exceeding the capacity. Each item can either be included or excluded, hence the name 0/1. Dynamic Programming solves this problem by breaking it into smaller subproblems and storing results in a table to avoid recomputation, ensuring an optimal solution.

Given:

- n items
- Each item has:
  - Weight w[i]
  - Profit/Value v[i]
- A knapsack with capacity W

- Total weight ≤ W
- Total profit is maximum
- Each item can be selected only once (0 or 1 time)

Algorithm / Steps:

Create

DP[n+1][W+1] for i

= 0 to n:

```
    for w = 0 to W:
        if i == 0 or w == 0:
            DP[i][w] = 0
        else if wt[i-1] <= w:

            DP[i][w] = max(

                        DP[i-1][w],
                        val[i-1] + DP[i-1][w - wt[i-1]]
                       )
        else:
            DP[i][w] = DP[i-1][w]

return DP[n][W]
```

Code:

```
#include <stdio.h>
#include <limits.h>

int max(int a, int b){
   return (a > b) ? a : b;
}

void printMatrix(int row, int col, int K[row][col]){
   for(int i = 0; i < row; i++){
      for(int j = 0; j < col; j++){
         printf("%d ", K[i][j]);
      }
      printf("\n");
   }
}

int main(){
   int n, amount;

   printf("Enter number of coin types: ");
   scanf("%d", &n);

   int D[n];
   printf("Enter coin denominations:\n");
   for(int i = 0; i < n; i++){
      scanf("%d", &D[i]);
   }

   printf("Enter amount: ");
   scanf("%d", &amount);

   int C[n][amount + 1];

   for(int i = 0; i < n; i++){
      C[i][0] = 0;
   }

   for(int i = 0; i < n; i++){
      for(int j = 1; j <= amount; j++){

         if(i == 0){
            if(j % D[i] == 0){
               C[i][j] = j / D[i];
            } else {
               C[i][j] = INT_MAX;
            }
         }
         else if(j < D[i]){
            C[i][j] = C[i-1][j];
```

```
            }
            else{
                int alt = (C[i][j - D[i]] == INT_MAX) ? INT_MAX : 1 + C[i][j - D[i]];
                C[i][j] = (C[i-1][j] < alt) ? C[i-1][j] : alt;
            }
        }
    }

    printMatrix(n, amount + 1, C);

    if(C[n-1][amount] == INT_MAX){
        printf("Not Possible\n");
    } else {
        printf("Minimum coins required: %d\n", C[n-1][amount]);
    }

    return 0;
}
```

Output:

```
PS C:\Users\viraj\OneDrive\Desktop\ADA\output> & .\'204_knapsack.exe'
Enter number of items: 4
Enter values of items: 12
10
20
15
Enter weights of items: 2
10
20
15
Enter weights of items: 2
20
15
Enter weights of items: 2
15
Enter weights of items: 2
Enter weights of items: 2
1
3
1
3
2
Enter capacity of knapsack: 5
3
2
Enter capacity of knapsack: 5
2
Enter capacity of knapsack: 5
0 0 0 0 0 0
0 0 0 12 12 12
12 12 0 10 12 22
0 0 0 0 0 0
0 0 0 12 12 12
12 12 0 10 12 22
22 22 22 0 10 12
12 12 0 10 12 22
22 22 22 0 10 12
12 22 30 32 0 10
22 22 22 0 10 12
12 22 30 32 0 10
12 22 30 32 0 10
Maximum profit: 37
Maximum profit: 37
Items included in the knapsack: 4 2 1
PS C:\Users\viraj\OneDrive\Desktop\ADA\output>
```

Practical 8

Title: Solve Given Problems Using Dynamic Programming: Matrix Chain Multiplication

Objective: To implement the Matrix Chain Multiplication question using the Dynamic Programming approach (matrix/table method)

Theory: Given a sequence of matrices, the goal is to find the most efficient way to multiply them. The order of multiplication affects the number of scalar multiplications, even though the result remains the same.

Goal: Find the minimum number of scalar multiplications needed. Let m[i][j] =

minimum cost of multiplying matrices from i to j Recurrence Relation:

$m[i,j] = \min_{i<k<j} \{m[i,k]+m[k+1,j]+p_{i-1}p_k p_j\}$

Algorithm / Steps:

- Start
- Input number of matrices n
- Input dimensions array $p[0 \dots n]$
- Declare matrix $m[n][n]$
- For $i = 1$to n
  Set $m[i][i] = 0$
- For chain length $L = 2$to n
  For $i = 1$to $n - L + 1$
  Set $j = i + L - 1$
  Set $m[i][j] = \infty$
  For $k = i$to $j - 1$
  Compute
  $q = m[i][k] + m[k + 1][j] + p[i - 1] \times p[k] \times p[j]$
  If $q < m[i][j]$
  Set $m[i][j] = q$
- Print $m[1][n]$as minimum number of multiplications
- Stop

Code:

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int matrixChainOrder(int p[], int n) {
    int m[MAX][MAX];

    for (int i = 1; i < n; i++) {
        m[i][i] = 0;
    }

    for (int L = 2; L < n; L++) {
        for (int i = 1; i < n - L + 1; i++) {
            int j = i + L - 1;
            m[i][j] = INT_MAX;

            for (int k = i; k <= j - 1; k++) {
                int q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (q < m[i][j]) {
                    m[i][j] = q;
                }
            }
        }
    }

    return m[1][n - 1];
}

int main() {
    int n;

    printf("Enter the number of matrices: ");
    scanf("%d", &n);

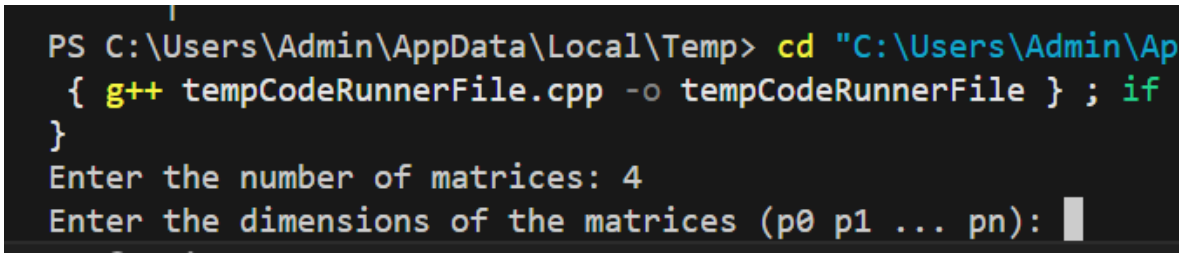
    int p[MAX];
    printf("Enter the dimensions of the matrices (p0 p1 ... pn): ");
    for (int i = 0; i <= n; i++) {
        scanf("%d", &p[i]);
    }

    int result = matrixChainOrder(p, n + 1);
    printf("Minimum number of multiplications is %d\n", result);

    return 0;
```

}

Output:


PS C:\Users\Admin\AppData\Local\Temp> cd "C:\Users\Admin\Ap
{ g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if
}
Enter the number of matrices: 4
Enter the dimensions of the matrices (p0 p1 ... pn):

Practical 9

Title: Implementation of LCS&SCS question using Dynamic Programming (Matrix Method)

Objective: To implement the LCS&SCS question using the Dynamic Programming approach (matrix/table method) and determine the Longest Common sequence and Shortest Common sequence

Theory: The Longest Common Subsequence (LCS) of two strings is the longest sequence of characters that appears in both strings in the same order, but not necessarily contiguously. The Shortest Common Subsequence (SCS) of two strings is the shortest string that contains both strings as subsequence.

SCS Length=m+n−LCS Length

Algorithm / Steps:

LCS

1. Create array dp[m+1][n+1]
2. For i = 0 to m dp[i][0] = 0
3. For j = 0 to n dp[0][j] = 0
4. For i = 1 to m For j = 1 to n

```
If X[i-1] == Y[j-1]
   dp[i][j] = 1 + dp[i-1][j-1]
Else
   dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

5. Return dp[m][n]

SCS

1. Create dp[m+1][n+1]
2. Initialize first row and column as 0
3. For i = 1 to m For j = 1 to n
   If X[i-1] == Y[j-1]
      dp[i][j] = 1 + dp[i-1][j-1]
   Else
      dp[i][j] = max(dp[i-1][j], dp[i][j-1])
4. Set i = m, j = n
5. While i > 0 and j > 0 If X[i-1] ==

```
Y[j-1]
    Add    X[i-1]    to
    result i--, j--
Else  if  dp[i-1][j] >  dp[i][j-1]
    Add X[i-1]
    i
--
El
se
    Add
    Y[j-1]
    j--
```

6. Add remaining characters of X (if any)
7. Add remaining characters of Y (if any)

Code:

```
#include<stdio.h>
#include<string.h>
int max(int a, int b)
{ return (a>b) ? a : b;
    }
int printMatrix(int row, int col, int C[row][col])
{ for(int i=0;i<row;i++){
            for(int j=0;j<col;j++){
                printf("%d ",C[i][j]);
            }
            printf("\n");
        }
    }
    int printLCS(int row, int col, int C[row][col], char X[row], char Y[col]){ int i=row-1,
        j=col-1;
        char lcs[100];
        int index=0;
        while(i>0 && j>0){
            if(X[i-1]==Y[j-
            1]){
                lcs[index++]=X[i-
                1]; i--;
                j--;
            }
            else if(C[i-1][j]>C[i][j-1]){ i--;
            }
            else{
                j--;
            }
        }
        lcs[index]='\0';
        printf("LCS: %s\n",strrev(lcs));
    }
    int printSCS(int row, int col, int C[row][col], char X[row], char Y[col]){ int i=row-1,
        j=col-1;
        char scs[200];
        int index=0;
        while(i>0 && j>0){
            if(X[i-1]==Y[j-
            1]){
                scs[index++]=X[i-
                1]; i--;
```

```
            j--;
        }
        else if(C[i-1][j]>C[i][j-1])
        {
            scs[index++]=X[i-
            1]; i--;
        }
        else{
            scs[index++]=Y[j-
            1]; j--;
        }
    }
    while(i>0){
        scs[index++]=X[i-
        1]; i--;
    }
    while(j>0){
        scs[index++]=Y[j-
        1]; j--;
    }
    scs[index]='\0';
    printf("SCS: %s\n",strrev(scs));
}
int main(){
    char X[100], Y[100];
    printf("Enter first string: "); scanf("%s", X);
    printf("Enter second string: "); scanf("%s",
    Y);
    int m = strlen(X);
    int n = strlen(Y);
    int C[m+1][n+1];
    for(int i=0;i<=m;i++){
        for(int j=0;j<=n;j++){
            if(i==0 || j==0){
                C[i][j]=0;
            }
            else if(X[i-1]==Y[j-1]){
                C[i][j]=1+C[i-1][j-1];
            }
            else{
                C[i][j]=max(C[i-1][j],C[i][j-1]);
            }
        }
    }
    printMatrix(m+1, n+1, C);
```

```
        printLCS(m+1, n+1, C, X, Y);
        printSCS(m+1, n+1, C, X, Y);

}
```

Output:

```
PS C:\Users\viraj\OneDrive\Desktop\ADA\output> & .\'204_LCS.exe'
Enter first string: acabaca
Enter second string: bacac
0 0 0 0 0 0
0 0 1 1 1 1
0 0 1 2 2 2
0 0 1 2 3 3
0 1 1 2 3 3
0 1 2 2 3 3
0 1 2 3 3 4
0 1 2 3 4 4
LCS: baca
SCS: acabacac
PS C:\Users\viraj\OneDrive\Desktop\ADA\output>
```

Practical 4

Title: Solve Given Problems using Greedy Strategy: Make a change problem

Objective: To solve making change problem using Greedy Strategy and Analyse it and compare it with Dynamic Approach

Theory: Given an amount of money and a set of coin denominations, the goal is to make change using the minimum number of coins.

At each step:

- Pick the largest denomination ≤ remaining amount
- Subtract it from the amount
- Repeat until amount becomes 0

Algorithm / Steps:

1. Sort coin denominations in descending order
2. Initialize remaining_amount = given amount
3. For each coin:
   While remaining_amount ≥ coin:
   - Select the coin
   - Subtract coin value from remaining_amount
4. Stop when remaining_amount = 0

Code:

```
#include <stdio.h>

int main() {
    int n, amount;

    printf("Enter number of coin denominations: ");
    scanf("%d", &n);

    int coins[n];

    printf("Enter coin denominations in descending order: ");
    for(int i = 0; i < n; i++) {
        scanf("%d", &coins[i]);
    }

    printf("Enter the amount to make change for: ");
    scanf("%d", &amount);

    int count = 0;

    for(int i = 0; i < n; i++) {
        while(amount >= coins[i]) {
            amount -= coins[i];
            count++;
        }
    }

    if(amount == 0) {
        printf("Minimum number of coins needed: %d\n", count);
    } else {
        printf("Cannot make change for the given amount with the provided denominations.\n");
    }

    return 0;
}
```

Output:

```
Enter number of coin denominations: 3
Enter coin denominations in descending order: 4
3
1
Enter the amount to make change for: 6
Minimum number of coins needed: 3
```

Practical 4

Title: Solve Given Problems using Greedy Strategy: Activity Selection

Objective: Solve the activity selection problem using Greedy Strategy.

Theory: Given n activities with start time and finish time, select the maximum number of non-overlapping activities that can be performed by a single person. Always select the activity that finishes earliest
Then select the next activity whose start time ≥ last selected finish time

Algorithm / Steps:

1. Sort all activities by increasing finish time
2. Select the first activity
3. For each remaining activity:

   If start_time ≥ finish_time of last selected activity
   → Select it

4. Repeat until all activities are checked

Code:

```
#include <stdio.h>

struct Activity {
   int start;
   int finish;
};

void sortActivities(struct Activity activities[], int n) {
   for(int i = 0; i < n - 1; i++) {
      for(int j = 0; j < n - i - 1; j++) {
         if(activities[j].finish > activities[j+1].finish) {
            struct Activity temp = activities[j];
            activities[j] = activities[j+1];
            activities[j+1] = temp;
         }
      }
   }
}

void selectActivities(struct Activity activities[], int n) {
   sortActivities(activities, n);

   printf("Selected activities:\n");
```

```
    int lastFinishTime = activities[0].finish;
    printf("Activity with start time %d and finish time %d\n",
           activities[0].start, activities[0].finish);

    for(int i = 1; i < n; i++) {
        if(activities[i].start >= lastFinishTime) {
            printf("Activity with start time %d and finish time %d\n",
                   activities[i].start, activities[i].finish);
            lastFinishTime = activities[i].finish;
        }
    }
}

int main() {
    int n;

    printf("Enter number of activities: ");
    scanf("%d", &n);

    struct Activity activities[n];

    printf("Enter start and finish times of activities:\n");
    for(int i = 0; i < n; i++) {
        printf("Activity %d: ", i + 1);
        scanf("%d %d", &activities[i].start, &activities[i].finish);
    }

    selectActivities(activities, n);

    return 0;
}
```

Output:

```
Enter number of activities: 6
Enter start and finish times of activities:
Activity 1: 1 2
Activity 2: 3 5
Activity 3: 4 6
Activity 4: 2 6
Activity 5: 5 9
Activity 6: 6 9
Selected activities:
Activity with start time 1 and finish time 2
Activity with start time 3 and finish time 5
Activity with start time 5 and finish time 9


=== Code Execution Successful ===
```

Practical 5

Title: Solve Given Problems using Greedy Strategy: Knapsack Problem

Objective: Solve Fractional Knapsack using Greedy Strategy and compare it with Dynamic Approach used for 0/1 Knapsack

Theory: The Knapsack Problem is a classic optimization problem solved using the Greedy Strategy in its fractional form. Given a set of items, each with a weight and a profit, and a knapsack with limited capacity, the objective is to maximize the total profit. In the Fractional Knapsack, items can be divided, allowing a fraction of an item to be taken. The greedy approach selects items based on the highest profit-to-weight ratio, ensuring maximum profit at each step. Items are chosen in decreasing order of this ratio until the knapsack is full.

Given:

- Capacity of knapsack = W
- Items with:
  - Profit pip_ipi
  - Weight wiw_iwi

Goal:

Maximize total profit by selecting items (fractions allowed). Select

items based on maximum profit per unit weight: $Ratio = p_i/w_i$

Pick items in descending order of ratio.

Algorithm / Steps:

1. Calculate p_i/w_i for all items
2. Sort items in descending order of ratio
3. Initialize:
   Total profit = 0
   Remaining capacity = W
4. For each item:
5. If weight ≤ capacity → take full item
6. Else → take fractional part
7. Stop when knapsack is full

Code:

```
#include <stdio.h>

struct Item {
    int value;
    int weight;
    float ratio;
};

void sortItems(struct Item items[], int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(items[j].ratio < items[j+1].ratio) {
                struct Item temp = items[j];
                items[j] = items[j+1];
                items[j+1] = temp;
            }
        }
    }
}

int main() {
    int n, capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item items[n];

    printf("Enter value and weight of items:\n");
    for(int i = 0; i < n; i++) {
        printf("Item %d: ", i + 1);
        scanf("%d %d", &items[i].value, &items[i].weight);
        items[i].ratio = (float)items[i].value / items[i].weight;
    }

    printf("Enter capacity of knapsack: ");
    scanf("%d", &capacity);

    sortItems(items, n);

    float totalValue = 0.0;

    for(int i = 0; i < n; i++) {
        if(items[i].weight <= capacity) {
            totalValue += items[i].value;
```

```
            capacity -= items[i].weight;
        } else {
            totalValue += items[i].ratio * capacity;
            break;
        }
    }

    printf("Maximum value in knapsack = %.2f\n", totalValue);

    return 0;
}
```

Output:

```
Enter number of items: 5
Enter value and weight of items:
Item 1: 1 20
Item 2: 2 40
Item 3: 3 10
Item 4: 4 40
Item 5: 5 60
Enter capacity of knapsack: 5
Maximum value in knapsack = 1.50
```

Practical 5

Title: Solve Given Problems using Greedy Strategy: Job Scheduling

Objective: To solve job scheduling problem using Greedy Strategy

Theory: The Job Scheduling Problem is a classic optimization problem solved using the Greedy Strategy. Given a set of jobs, each with a deadline and profit, where each job takes one unit of time, the objective is to maximize total profit by scheduling jobs within their deadlines. The greedy approach selects jobs in decreasing order of profit and assigns each job to the latest available time slot before its deadline. This ensures that high-profit jobs are given priority while maintaining feasibility of the schedule.

Given:

- n jobs with:
  - Deadline d_i
  - Profit p_i
- Each job takes 1 unit of time

Goal: Maximize total profit by scheduling jobs within deadlines. Select jobs

based on maximum profit first

- Sort jobs in descending order of profit
- Assign each job to the latest possible free slot before its deadline

Algorithm / Steps:

1. Sort all jobs in decreasing order of profit
2. Find maximum deadline → create slots array
3. For each job:
   Check from its deadline to 1 Place
   job in the latest free slot
4. Calculate total profit

Code:

```
#include <stdio.h>

struct Job {
    int deadline;
    int profit;
};

void sortJobs(struct Job jobs[], int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(jobs[j].profit < jobs[j+1].profit) {
                struct Job temp = jobs[j];
                jobs[j] = jobs[j+1];
                jobs[j+1] = temp;
            }
        }
    }
}

int main() {
    int n;

    printf("Enter number of jobs: ");
    scanf("%d", &n);

    struct Job jobs[n];

    printf("Enter deadline and profit of jobs:\n");
    for(int i = 0; i < n; i++) {
        printf("Job %d: ", i + 1);
        scanf("%d %d", &jobs[i].deadline, &jobs[i].profit);
    }

    sortJobs(jobs, n);

    int maxDeadline = 0;
    for(int i = 0; i < n; i++) {
        if(jobs[i].deadline > maxDeadline) {
            maxDeadline = jobs[i].deadline;
        }
    }

    int schedule[maxDeadline];
    for(int i = 0; i < maxDeadline; i++) {
        schedule[i] = -1;
```

```
    }

    int totalProfit = 0;

    for(int i = 0; i < n; i++) {
        for(int j = jobs[i].deadline - 1; j >= 0; j--) {
            if(j < maxDeadline && schedule[j] == -1) {
                schedule[j] = i;
                totalProfit += jobs[i].profit;
                break;
            }
        }
    }

    printf("Total profit: %d\n", totalProfit);
    printf("Scheduled jobs:\n");

    for(int i = 0; i < maxDeadline; i++) {
        if(schedule[i] != -1) {
            printf("Job with deadline %d and profit %d\n",
                   jobs[schedule[i]].deadline,
                   jobs[schedule[i]].profit);
        }
    }

    return 0;
}
```

Output:

```
Enter number of jobs: 4
Enter deadline and profit of jobs:
Job 1: 1 34
Job 2: 4 57
Job 3: 2 44
Job 4: 2 10
Total profit: 135
Scheduled jobs:
Job with deadline 1 and profit 34
Job with deadline 2 and profit 44
Job with deadline 4 and profit 57
```

Practical 6

Title: Solve Given Problems using Greedy Strategy: MST using Prim's Algorithm

Objective: To make Minimum Spanning Tree using Prim's Algorithm

Theory: A Minimum Spanning Tree (MST) is a subset of edges of a connected, weighted, undirected graph that:

- Connects all vertices
- Has no cycles
- Has minimum total edge weight

Prim's algorithm builds the MST by:

- Starting from any vertex
- Repeatedly adding the minimum weight edge that connects a vertex inside the tree to a vertex outside

It always makes the locally optimal (greedy) choice

Algorithm / Steps:

1. Start with any vertex (say vertex 0)
2. Mark it as visited
3. Repeat until all vertices are included:
    - Select the minimum weight edge connecting:
        - a visited vertex → an unvisited vertex
    - Add this edge to MST
    - Mark the new vertex as visited

Code:

```
#include <stdio.h>
#include <limits.h>

#define MAX_VERTICES 100

int graph[MAX_VERTICES][MAX_VERTICES];

void primMST(int vertices) {
   int parent[MAX_VERTICES];
   int key[MAX_VERTICES];
   int mstSet[MAX_VERTICES];

   for (int i = 0; i < vertices; i++) {
      key[i] = INT_MAX;
      mstSet[i] = 0;
   }

   key[0] = 0;
   parent[0] = -1;

   for (int count = 0; count < vertices - 1; count++) {
      int minKey = INT_MAX;
      int minIndex = -1;

      for (int v = 0; v < vertices; v++) {
         if (mstSet[v] == 0 && key[v] < minKey) {
            minKey = key[v];
            minIndex = v;
         }
      }

      if(minIndex == -1) break;

      mstSet[minIndex] = 1;

      for (int v = 0; v < vertices; v++) {
         if (graph[minIndex][v] && mstSet[v] == 0 && graph[minIndex][v] < key[v]) {
            parent[v] = minIndex;
            key[v] = graph[minIndex][v];
         }
      }
   }

   printf("Edge \tWeight\n");
   for (int i = 1; i < vertices; i++) {
      printf("%d - %d \t%d\n", parent[i], i, graph[parent[i]][i]);
   }
}
```

```
int main() {
    int vertices, edges;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    for(int i = 0; i < vertices; i++) {
        for(int j = 0; j < vertices; j++) {
            graph[i][j] = 0;
        }
    }

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    for (int i = 0; i < edges; i++) {
        int u, v, weight;
        printf("Enter edge (u v weight): ");
        scanf("%d %d %d", &u, &v, &weight);

        u--;
        v--;

        graph[u][v] = weight;
        graph[v][u] = weight;
    }

    primMST(vertices);

    return 0;
}
```

Output:

```
PS C:\Users\Admin\AppData\Local\Temp> cd "C:\Users\A
 { gcc tempCodeRunnerFile.c -o tempCodeRunnerFile }
Enter number of vertices: 7
Enter number of edges: 9
Enter edge (u v weight): 0 1 18
Enter edge (u v weight): 3 4 5
Enter edge (u v weight): 4 5 7
Enter edge (u v weight): 23 5 7
Enter edge (u v weight): 54 6 8
Enter edge (u v weight): 35 8 6
Enter edge (u v weight): 3 5 0
Enter edge (u v weight): 3 5 7
Enter edge (u v weight): 34 6 8
Edge    Weight
0 - 1   18
0 - 2   0
0 - 3   0
0 - 4   0
0 - 5   0
0 - 6   0
PS C:\Users\Admin\AppData\Local\Temp>
```

Practical 6

Title: Solve Given Problems using Greedy Strategy: MST using Krushkal's Algorithm
Objective: To make Minimum Spanning Tree using Krushkal's Algorithm

Theory: A Minimum Spanning Tree (MST) is a subset of edges of a connected, weighted, undirected graph that:

- Connects all vertices
- Has no cycles
- Has minimum total edge weight

Kruskal's algorithm is a Greedy Algorithm used to find the Minimum Spanning Tree (MST) by:

- Selecting edges in increasing order of weight
- Ensuring no cycle is formed

Always pick the smallest weight edge that does not form a cycle

Algorithm / Steps:

1. Sort all edges in ascending order of weight
2. Initialize MST as empty
3. For each edge:
    - If adding the edge does not form a cycle, include it in MST
    - Otherwise, discard it
4. Stop when (V - 1) edges are selected

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_EDGES 100

struct Edge {
   int src, dest, weight;
};

struct subset {
   int parent;
   int rank;
};

int find(struct subset subsets[], int i) {
   if (subsets[i].parent != i)
      subsets[i].parent = find(subsets, subsets[i].parent);
   return subsets[i].parent;
}

void unionSets(struct subset subsets[], int x, int y) {
   int rootX = find(subsets, x);
   int rootY = find(subsets, y);

   if (subsets[rootX].rank < subsets[rootY].rank)
      subsets[rootX].parent = rootY;
   else if (subsets[rootX].rank > subsets[rootY].rank)
      subsets[rootY].parent = rootX;
   else {
      subsets[rootY].parent = rootX;
      subsets[rootX].rank++;
   }
}

int compareEdges(const void* a, const void* b) {
   struct Edge* edgeA = (struct Edge*)a;
   struct Edge* edgeB = (struct Edge*)b;
   return edgeA->weight - edgeB->weight;
}

void kruskalMST(struct Edge edges[], int V, int E) {
   struct subset* subsets = (struct subset*)malloc(V * sizeof(struct subset));
```

```
    for (int v = 0; v < V; v++) {
        subsets[v].parent = v;
        subsets[v].rank = 0;
    }

    qsort(edges, E, sizeof(struct Edge), compareEdges);

    struct Edge result[MAX_EDGES];
    int e = 0;
    int i = 0;

    while (e < V - 1 && i < E) {
        struct Edge nextEdge = edges[i++];

        int x = find(subsets, nextEdge.src);
        int y = find(subsets, nextEdge.dest);

        if (x != y) {
            result[e++] = nextEdge;
            unionSets(subsets, x, y);
        }
    }

    printf("Edge \tWeight\n");
    for (int i = 0; i < e; i++) {
        printf("%d - %d \t%d\n", result[i].src, result[i].dest, result[i].weight);
    }

    free(subsets);
}

int main() {
    int V, E;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter number of edges: ");
    scanf("%d", &E);

    struct Edge edges[MAX_EDGES];

    for (int i = 0; i < E; i++) {
        printf("Enter edge (src dest weight): ");
        scanf("%d %d %d", &edges[i].src, &edges[i].dest, &edges[i].weight);
```

```
    }

    kruskalMST(edges, V, E);

    return 0;
}
```

Output:

```
PS C:\Users\Admin\AppData\Local\Temp> cd
 { gcc tempCodeRunnerFile.c -o tempCodeR
Enter number of vertices: 7
Enter number of edges: 8
Enter edge (src dest weight): 2
33 4
Enter edge (src dest weight): 2 4 5
Enter edge (src dest weight): 34 4
5
Enter edge (src dest weight): 3 6 7
Enter edge (src dest weight): 20 10 9
Enter edge (src dest weight): 2 7 9
Enter edge (src dest weight): 2 4 0
Enter edge (src dest weight): 1 44 6
PS C:\Users\Admin\AppData\Local\Temp> W
```

Practical 10

Title: Solve following problems using Backtracking 4-Queen Problem

Objective: Solve the 4-Queen Problem using DFS i.e. Backtracking

Theory: Place 4 queens on a 4 × 4 chessboard such that:

- No two queens attack each other
- i.e., no same row, column, or diagonal

Backtracking tries to place queens row by row and:

- Checks if placing a queen is safe
- If not safe → backtrack and try next column

Algorithm / Steps:

1. Start from first row
2. For each column in the current row:
   a. Check if position is safe
   b. If safe:
      Place queen
      Recur for next row
   c. If not safe:
      Try next column
3. If all queens are placed then print solution
4. If no column works then backtrack

Code:

```
#include <stdio.h>

#define N 4

int board[N][N] = {0};

int isSafe(int row, int col) {
   for (int i = 0; i < col; i++) {
      if (board[row][i]) return 0;
   }

   for (int i = row, j = col; i >= 0 && j >= 0; i--, j--) {
      if (board[i][j]) return 0;
   }

   for (int i = row, j = col; i < N && j >= 0; i++, j--) {
      if (board[i][j]) return 0;
   }

   return 1;
}

int solveNQUtil(int col) {
   if (col >= N) return 1;

   for (int i = 0; i < N; i++) {
      if (isSafe(i, col)) {
         board[i][col] = 1;

         if (solveNQUtil(col + 1)) return 1;

         board[i][col] = 0;
      }
   }

   return 0;
}

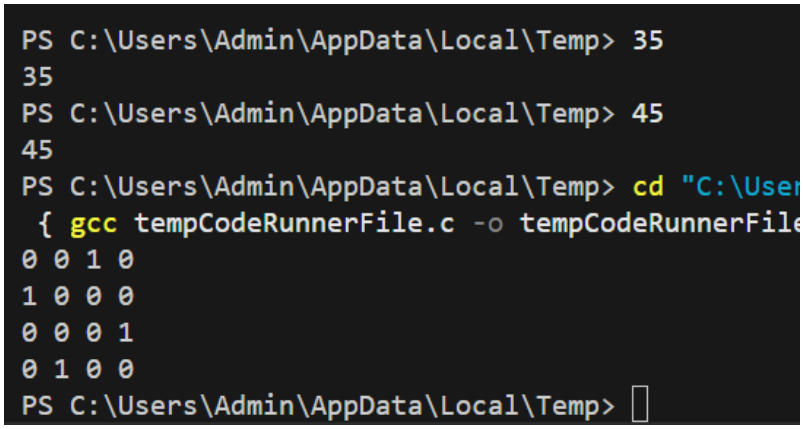
void solveNQ() {
   if (solveNQUtil(0) == 0) {
      printf("Solution does not exist\n");
      return;
   }

   for (int i = 0; i < N; i++) {
      for (int j = 0; j < N; j++) {
         printf("%d ", board[i][j]);
      }
      printf("\n");
   }
```

```
}

int main() {
   solveNQ();
   return 0;
}
```

Output:



Practical 10

Title: Solve following problems using Backtracking 8-Queen Problem

Objective: Solve the 8-Queen Problem using DFS i.e. Backtracking

Theory: Place 8 queens on an 8 × 8 chessboard such that:

- No two queens attack each other
- No same row, column, or diagonal

Backtracking Place queens row by row

- Try each column in the current row
- If safe → place queen
- Else → try next column
- If no column works → backtrack

Algorithm / Steps:

1. Start from row = 0
2. For each column in current row:
   a. Check if safe
   b. If safe:
      Place Queen
      Recur next row
   c. If not safe:
      Try next column
3. If row == 8 then solution found
4. If all column fail then backtrack

Code:

```
#include <stdio.h>

#define N 8

int board[N][N] = {0};

int isSafe(int row, int col) {
   for (int i = 0; i < col; i++) {
      if (board[row][i]) return 0;
   }

   for (int i = row, j = col; i >= 0 && j >= 0; i--, j--) {
      if (board[i][j]) return 0;
   }

   for (int i = row, j = col; i < N && j >= 0; i++, j--) {
      if (board[i][j]) return 0;
   }

   return 1;
}

int solveNQUtil(int col) {
   if (col >= N) return 1;

   for (int i = 0; i < N; i++) {
      if (isSafe(i, col)) {
         board[i][col] = 1;

         if (solveNQUtil(col + 1)) return 1;

         board[i][col] = 0;
      }
   }

   return 0;
```

```
}

void solveNQ() {
   if (solveNQUtil(0) == 0) {
      printf("Solution does not exist\n");
      return;
   }

   for (int i = 0; i < N; i++) {
      for (int j = 0; j < N; j++) {
         printf("%d ", board[i][j]);
      }
      printf("\n");
   }
}

int main() {
   solveNQ();
   return 0;
}
```

Output:

```
PS C:\Users\Admin\AppData\Local\Temp> cd "C:\Us
 { gcc tempCodeRunnerFile.c -o tempCodeRunnerFi
1 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0
PS C:\Users\Admin\AppData\Local\Temp>
```

Practical 3

Title: Perform following sorting operation and measure the execution time for sufficient large input: Merge Sort, Quick Sort

Objective: To implement Merge sort and Quick Sort algorithms on large datasets, measure their execution time, and analyze their efficiency based on time complexity.

Theory: Sorting is the process of arranging data elements in a specific order. Merge Sort and Quick Sort are efficient comparison-based sorting algorithms based on the divide-and-conquer approach. **Merge Sort** divides the array into two halves, recursively sorts each half, and then merges the sorted halves to produce the final sorted array. It guarantees a stable sort and consistent performance. **Quick Sort** selects a pivot element and partitions the array into two subarrays such that elements less than the pivot are placed on one side and greater elements on the other. The same process is then applied recursively to the subarrays. **Merge Sort** has a time complexity of O(n log n) in all cases but requires additional memory for merging. Quick Sort has an average time complexity of O(n log n) and is faster in practice, but its worst-case time complexity is $O(n^2)$ if poor pivot selection occurs. Both algorithms are more efficient than simple sorting methods for large datasets.

Algorithm / Steps:

Quick Sort

```
QUICKSORT(A, low, high)
1. if low < high
2.     p = PARTITION(A, low, high)

3.     QUICKSORT(A, low, p-1)
4.     QUICKSORT(A, p+1, high)

 PARTITION(A, low, high)
1. pivot = A[high]
2. i = low - 1
3. for j = low to high-1
4.       if A[j] < pivot
5.        i++
6.        swap A[i] and A[j]
7. swap A[i+1] and A[high]
8. return i+1
```

Merge Sort

```
MERGE-SORT(A, low, high)
1. if low < high
2. mid = (low + high) / 2
3. MERGE-SORT(A, low, mid)
4. MERGE-SORT(A, mid + 1, high)
5. MERGE(A, low, mid, high)

MERGE(A, low, mid, high)
1. create two temporary arrays: L
```

= A[low to mid]
R = A[mid+1 to high]
2. i = 0, j = 0, k = low
3. while i < size(L) and j < size(R)
4. if L[i] ≤ R[j]
5. A[k] = L[i]
6. i++
7. else
8. A[k] = R[j]
9. j++
10. k++
11. copy remaining elements of L (if any)
12. copy remaining elements of R (if any)

Code:

```
//Quick Sort
#include <stdio.h>

    void swap(int arr[], int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    int partition(int arr[], int low, int high) {
        int pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                swap(arr, i, j);
            }
        }

        swap(arr, i + 1, high);
        return i + 1;
    }

    void quickSort(int arr[], int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

    int main() {
        int arr[] = {14, 3, 19, 7, 2, 11, 6, 18, 1, 10, 5, 15, 9, 4, 13};
        int n = sizeof(arr) / sizeof(arr[0]);

        quickSort(arr, 0, n - 1);

        printf("Sorted array: ");
        for (int i = 0; i < n; i++) {
            printf("%d ", arr[i]);
        }

        printf("\n");
        return 0;
    }
```

```
//Merge Sort
    #include <stdio.h>

    void merge(int arr[], int l, int m, int r) {
       int i, j, k;
       int n1 = m - l + 1;
       int n2 = r - m;

       int L[n1], R[n2];

       for (i = 0; i < n1; i++)
          L[i] = arr[l + i];

       for (j = 0; j < n2; j++)
          R[j] = arr[m + 1 + j];

       i = 0;
       j = 0;
       k = l;

       while (i < n1 && j < n2) {
          if (L[i] <= R[j]) {
             arr[k] = L[i];
             i++;
          } else {
             arr[k] = R[j];
             j++;
          }
          k++;
       }

       while (i < n1) {
          arr[k] = L[i];
          i++;
          k++;
       }

       while (j < n2) {
          arr[k] = R[j];
          j++;
          k++;
       }
    }

    void mergeSort(int arr[], int l, int r) {
       if (l < r) {
          int m = l + (r - l) / 2;

          mergeSort(arr, l, m);
          mergeSort(arr, m + 1, r);
```

```
        merge(arr, l, m, r);
    }
}

int main() {
    int arr[] = {14, 3, 19, 7, 2, 11, 6, 18, 1, 10, 5, 15, 9, 4, 13};
    int n = sizeof(arr) / sizeof(arr[0]);

    mergeSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    printf("\n");
    return 0;
}
```

Output:

```
PS C:\Users\Admin\AppData\Local\Temp> cd "C:\Users\Admin\AppData\
 { gcc tempCodeRunnerFile.c -o tempCodeRunnerFile } ; if ($?) { .
Sorted array: 1 2 3 4 5 6 7 9 10 11 13 14 15 18 19
PS C:\Users\Admin\AppData\Local\Temp>
```

Merge Sort

```
PS C:\Users\Admin\AppData\Local\Temp> cd "C:\Users\Admin\AppData\Local\Te
 { gcc tempCodeRunnerFile.c -o tempCodeRunnerFile } ; if ($?) { .\tempCod
Sorted array: 1 2 3 4 5 6 7 9 10 11 13 14 15 18 19
PS C:\Users\Admin\AppData\Local\Temp>
```

V.V.P. ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER ENGINEERING

Semester: 4th A.Y.:2025-26(EVEN)

Subject: Object Oriented ProgrammingBE04000231)

Student Name:_____ **Class:**_____

Enrollment Number:_____ **Batch:**_____

INDEX

Sr. No.	Name of Experiment	Date	Faculty Sign	Remarks
1	1.1 Write a program to display first n prime numbers.			
	1.2 Write a program that takes the lengths of three sides of a triangle as input. Calculate and print the area of the triangle. Ensure that the program validates if the given side lengths can actually form a triangle (sum of any two sides must be greater than the third side) before calculating the area (use Heron's formula: Area = sqrt(s*(s-a) *(s-b) *(s-c)) where s = (a+b+c)/2).			
2	2.1 Write a Java program that prompts the user to enter a single letter (character). Determine whether the entered character is a vowel (a, e, i, o, u, case-insensitive) or a consonant, and display the result.			
	2.2 Write a Java program that accepts a five-digit integer from the keyboard. Your program should then create a new number by adding one to each digit of the input number. For example, if the input is 12391, the output should be 23402 (note: 9+1=10, so it becomes 0 with a carry).			
3	3.1 Write a program to validate email address.			
	3.2 Write a program to validate GTU enrollment no.			
	3.3 Write a program to check given Strings are anagram or not.			
	4.1 Define a class Student with name of student, department, and 3 subject marks as data members. Define methods to enter details of 5 students, print details of the entered students and sort details of the students on the base of their percentage.			

4	4.2 Create an Employee class with private instance variables for employeeName (String) and employeeSalary (double). Implement public methods readEmployeeData() (to take input from the user) and displayEmployeeData() (to print the employee's name and salary).Demonstrate object creation and method invocation in a main method.			
	4.3 Design a class Time with hours (int) and minutes (int) as data members. Include method setTime(int h, int m) to initialize the time and displayTime() to display the time. Implement a method addTime(Time t1, Time t2) that takes two Time objects as arguments, adds their hours and minutes, and stores the result in the calling object. Do not use constructors for initialization in this specific practical.			
5	5.1 Create a Java class named University with a static data member totalStudents to keep track of the number of student objects created. Implement a static method getTotalStudents(). Also, include a static block to initialize a static variable (e.g., universityName) and an instance block to print a message when an object is created. Demonstrate their execution order.			
	5.2 Write a Java program that demonstrates method overloading to calculate the volume of different 3D shapes. Implement overloaded methods named calculateVolume() for a Cube (takes one side length), a RectangularCube (takes length, width, height), and a Sphere (takes radius).			
6	6.1 Write a Java program to demonstrate method overriding in an online payment system. Create a superclass Payment with a method processPayment(int amount). In the subclass CreditCardPayment, override the method to print "Payment of Rs.<amount> done using Credit Card". In the subclass UPIPayment, override the method to print "Payment of Rs.<amount> done using UPI". Accept user choice and amount, and display the payment result using method overriding.			

	6.2 Create a base class named Employee that contains a method displayDetails() which prints general employee details such as name and department. Now create a subclass Manager that inherits from Employee and overrides the displayDetails() method to include additional information such as the manager's team size or project name. In the main method, create objects of both Employee and Manager classes and call the displayDetails() method using each object to show how Java determines which version of the method to execute at runtime.			
7	7.1 Describe abstract class called Shape which has three subclasses say Triangle, Rectangle and Circle. Define one method area() in the abstract class and override this area() in these three subclasses to calculate for specific object.			
	7.2 Write a program to create interface and declare one method interest(p,r,n) then create two class SimpleInterest and Compound Interest and calculate simple and compound interest.			
8	8.1 Write a program to create package and import package.			

9	9.1 Take the value of denominator and numerator from user using command-line argument. Implement the concept of exception handling to manage all possible run-time error.			
	9.2 Write a program to demonstrate throw and throws keyword.			
10	10.1 Design PrettyPrinter in a nonprimitive manner using Thread.			
	10.2 Write a Java program that creates two threads: ● First thread prints numbers from 1 to 10 at the interval of 1 second. ● Second thread prints numbers from 11 to 20 at the interval of 500 ms. Run both threads and display the output.			

11	11.1 Write a program to export book data in csv format.			
	11.2 Implement file read, write and append operation.			
12	12.1 Create ArrayList mover. In which selected element of one list moved to another. i.e. Subject Selected Subject Java DAA Maths-I EME Database Lets move Java and DAA to selected subject Subject Selected Subject Maths-I Java EME DAA Database			
	12.2 Create a generic class Box<T> with a method addItem (T item) that stores items in an ArrayList<T>. Demonstrate the class by: ● Creating a Box<String> for names. ● Creating a Box<Integer> for roll numbers. ● Display stored items for both.			
13	13.1 Write a javafx form for user signup which includes username(Textfield), password(passwordfield), gender(radiobutton),branch(combobox),address(textarea),hobbies(checkbox). Create two buttons display value of user data into a label. signup(button), clear(button)			
	13.2 Write a javafx program to move a circle from left to right when navigation keys been clicked. When mouse hovered over the circle then it change the colour of circle.			

Course Outcomes	Students will be able to
CO1	Describe the structure of Java programs and the foundational elements of the Java programming language, including data types, operators, control structures, memory model, and execution environment.
CO2	Apply Object-Oriented Programming principles such as encapsulation, inheritance, abstraction, and polymorphism to design modular and reusable Java applications using classes, constructors, interfaces, and packages.
CO3	Implement exception handling, multithreading, and file I/O operations to develop robust and efficient Java applications.
CO4	Develop GUI-based applications using JavaFX with proper event handling and layout management.
CO5	Utilize Java Collections and Generics to store, retrieve, and process data efficiently in Java programs.

Faculty

Prof. Brinda Chanv

Prof. Bhavesh Chhatrala

Prof. Gunjani Vaghela

H.O.D.

Dr.M.V.Dhamecha

PRACTICAL – 5

Aim :

Describe abstract class called Shape which has three subclasses say Triangle, Rectangle and Circle. Define onemethod area() in the abstract class and override this area() in these three subclasses to calculate for specific object.

CODE :

```
// Abstract class abstract
class Shape {
// Abstract method
abstract void area(); }
// Triangle subclass class
Triangle extends Shape
{     double base, height;

   Triangle(double b, double h) {
      base = b;
      height = h;
   }

   void area() {
      double result = 0.5 * base * height;
      System.out.println("Area of Triangle = " + result);
   }
}

// Rectangle subclass class
Rectangle extends Shape {
double length, breadth;

   Rectangle(double l, double b) {
length = l;
      breadth = b;
   }

   void area() {
      double result = length * breadth;
      System.out.println("Area of Rectangle = " + result);
   }
}

// Circle subclass
class Circle extends
Shape {     double
radius;

   Circle(double r) {
```

```
        radius = r;
    }

    void area() {         double result = Math.PI *
radius * radius;          System.out.println("Area of
Circle = " + result);
    }
}

// Main class public class Main {
public static void main(String[] args) {
Shape t = new Triangle(5, 4);
        Shape r = new Rectangle(6, 3);
Shape c = new Circle(2.5);

        t.area();
        r.area();
        c.area();
    }
}
```

OUTPUT :

```
Area of Triangle = 10.0
Area of Rectangle = 18.0
Area of Circle = 19.634954084936208
```

PRACTICAL – 5

AIM :

Write a program to create interface and declare one method interest (p,r,n) then create two class Simple Interest and Compound Interest and calculate simple and compound interest.

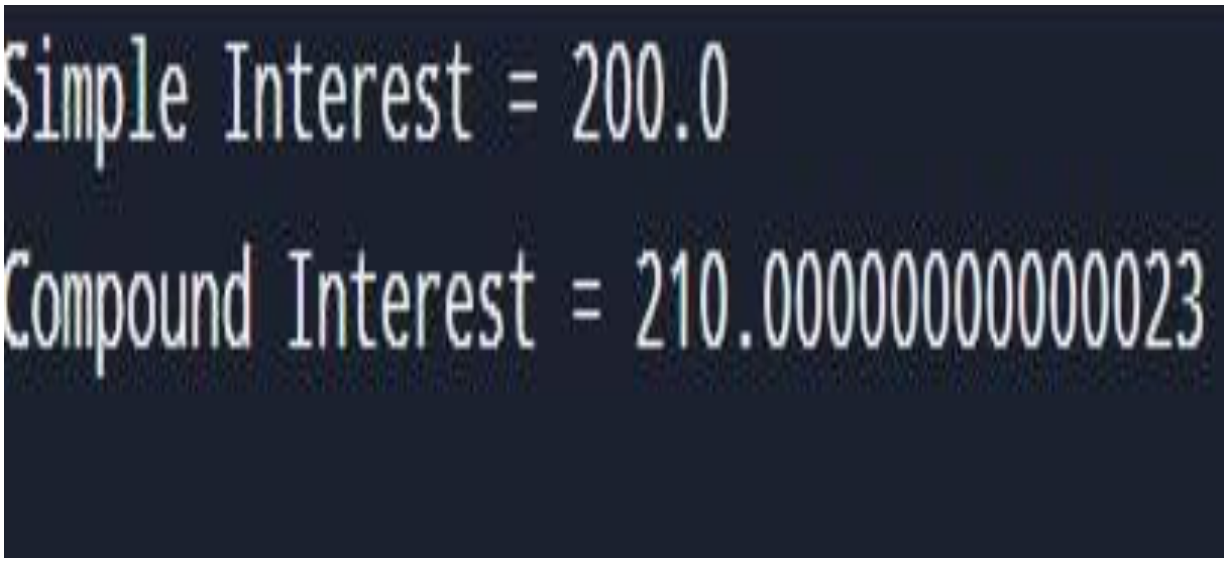
CODE :

```
// Interface interface
Interest {
   void interest(double p, double r, double n);
}
// Class for Simple Interest
class SimpleInterest implements Interest {     public
void interest(double p, double r, double n) {         double
si = (p * r * n) / 100;
       System.out.println("Simple Interest = " + si);
   }
}
// Class for Compound Interest class
CompoundInterest implements Interest {     public void
interest(double p, double r, double n) {           double
amount = p * Math.pow((1 + r / 100), n);        double ci
= amount - p;
       System.out.println("Compound Interest = " + ci);
   }
}
// Main class public class Main {     public
static    void    main(String[]    args)    {
double p = 1000, r = 10, n = 2;

       Interest si = new SimpleInterest();        Interest
ci = new CompoundInterest();

       si.interest(p, r, n);
       ci.interest(p, r, n);
   }
}
```

OUTPUT :


Simple Interest = 200.0
Compound Interest = 210.00000000000023

PRACTICAL– 5

AIM :

Write a program to create package and import package.

CODE :

```
// Simulating package (comment only for understanding) //
package MyPackage;

// Class inside package class
Addition {        public void
add(int a, int b) {        int sum
= a + b;
        System.out.println("Sum = " + sum);
    }
}

// Main class (simulating import) public class
Main {        public static void main(String[]
args) {        // No need of import in same file
Addition obj =
new Addition();
        obj.add(10, 20);
    }
}
```

OUTPUT :



PRACTICAL – 6

AIM :

Take the value of denominator and numerator from user using command-line argument. Implement the concept of exception handling to manage all possible run-time error.

CODE :

```
public class Main {
    public static void main(String[] args) {

        // Simulated command-line arguments        args
= new String[]{"10", "2"};

        try {
            int numerator = Integer.parseInt(args[0]);
int denominator = Integer.parseInt(args[1]);            int
result = numerator / denominator;

            System.out.println("Result = " + result);
        }
        catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero");
        }
        catch (NumberFormatException e) {
            System.out.println("Error: Invalid number format");
        }
        catch     (ArrayIndexOutOfBoundsException     e)     {
System.out.println("Error: Insufficient arguments");
        }
        catch (Exception e) {
            System.out.println("General Error: " + e);
        }
    }
}
```

OUTPUT :

```
Result = 5

=== Code Execution Successful ===
```

```
ERROR!
Error: Cannot divide by zero

=== Code Execution Successful ===
```

```
ERROR!
Error: Invalid number format
```

```
ERROR!
Error: Insufficient arguments
```

PRACTICAL – 6

AIM :

Write a program to demonstrate throw and throws keyword.

CODE :

```
import java.util.Scanner;

class Test {

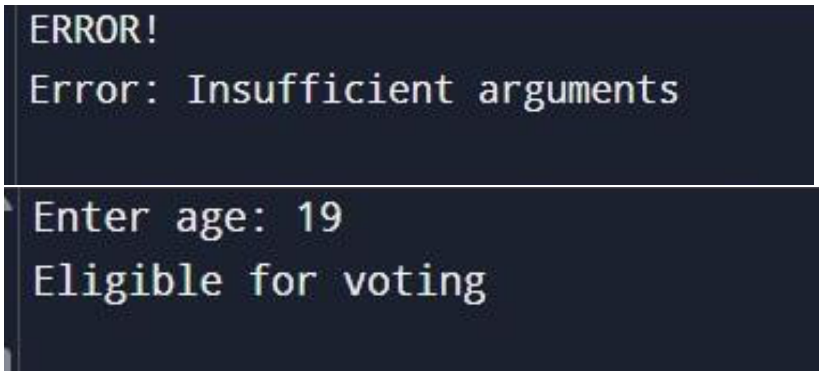
 // Method using 'throws'

 static void checkAge(int age) throws ArithmeticException {
if (age < 18) {             // using 'throw'
            throw new ArithmeticException("Not eligible for voting");
        } else {
            System.out.println("Eligible for voting");
        }
    }
} public class Main
{
    public static void main(String[] args) {
        try {
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter age: ");           int
age = sc.nextInt();

            // calling method
            Test.checkAge(age);
        }
        catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }
}
```

OUTPUT :


ERROR!
Error: Insufficient arguments
Enter age: 19
Eligible for voting

PRACTICAL – 7

AIM :

Design Pretty Printer in a nonprimitive manner using Thread.

CODE :

```
class PrettyPrinter {

   // synchronized method (non-primitive synchronization)
public synchronized void print(String msg) {
      System.out.print("[ ");
      try {
         Thread.sleep(500); // simulate delay
      } catch (InterruptedException e) {
         System.out.println(e);
      }
      System.out.println(msg + " ]");
   }
}

// Thread class
class MyThread extends Thread {
   PrettyPrinter          printer;
String message;

   MyThread(PrettyPrinter p, String msg) {       printer
= p;
      message = msg;
   }

   public void run() {
      printer.print(message);
   }
}
// Main class public class Main {    public
static void main(String[] args) {

      PrettyPrinter printer = new PrettyPrinter();

      MyThread t1 = new MyThread(printer, "Hello");
      MyThread t2 = new MyThread(printer, "Welcome");
MyThread t3 = new MyThread(printer, "Java");
t1.start();
t2.start();
t3.start();
```

```
    }
}
```

OUTPUT :



PRACTICAL – 7

AIM :

Write a Java program that creates two threads:

- First thread prints numbers from 1 to 10 at the interval of 1 second.
- Second thread prints numbers from 11 to 20 at the interval of 500 ms.Run both threads and display the output.

• CODE :

```
class Thread1 extends Thread {
public void run() {
      try {
         for (int i = 1; i <= 10; i++) {
            System.out.println("Thread 1: " + i);
            Thread.sleep(1000); // 1 second
         }
      } catch (InterruptedException e) {
         System.out.println(e);
      }
   }
}

class Thread2 extends Thread {
public void run() {
      try {
         for (int i = 11; i <= 20; i++) {
            System.out.println("Thread 2: " + i);
            Thread.sleep(500); // 500 ms
         }
      } catch (InterruptedException e) {
         System.out.println(e);
      }
   }
}

public class Main {     public static void
main(String[] args) {
      Thread1 t1 = new Thread1();
      Thread2 t2 = new Thread2();

      t1.start();
t2.start();
   }
```

}

OUTPUT :

```
Thread 1: 1
Thread 2: 11
Thread 2: 12
Thread 1: 2
Thread 2: 13
Thread 2: 14
Thread 1: 3
Thread 2: 15
Thread 2: 16
Thread 1: 4
Thread 2: 17
Thread 2: 18
Thread 1: 5
Thread 2: 19
Thread 2: 20
Thread 1: 6
Thread 1: 7
Thread 1: 8
Thread 1: 9
Thread 1: 10
```

PRACTICAL- 8

AIM :

Write a program to export book data in csv format.

CODE :

```
import java.io.FileWriter; import
java.io.IOException;

class Book {
   int id;
String title;
String author;
double price;
Book(int id, String
title, String author,
double price) {
this.id = id;
      this.title = title;       this.author
= author;
      this.price = price;
   }
}

public class Main {
   public static void main(String[] args) {

      // Book data
      Book b1 = new Book(1, "Java Basics", "James", 250);
      Book b2 = new Book(2, "Python Guide", "Guido",
300);      Book b3 = new Book(3, "C++ Mastery", "Bjarne",
400);

      try {
         FileWriter fw = new FileWriter("books.csv");

         // CSV Header
         fw.write("ID,Title,Author,Price\n");

         // Writing data          fw.write(b1.id + "," + b1.title + "," + b1.author
+ "," + b1.price + "\n");          fw.write(b2.id + "," + b2.title + "," + b2.author
+ "," + b2.price + "\n");          fw.write(b3.id + "," + b3.title + "," + b3.author
+ "," + b3.price + "\n");          fw.close();
```

```
            System.out.println("Data exported to books.csv successfully!");
        } catch (IOException e) {
            System.out.println("Error: " + e);
        }
    }
}
```

OUTPUT :

```
Data exported to books.csv successfully!
```

```
ID,Title,Author,Price
1,Java Basics,James,250.0
2,Python Guide,Guido,300.0
3,C++ Mastery,Bjarne,400.0
```

PRACTICAL – 08

AIM :

Implement file read, write and append operation.

CODE :

```
import java.io.*; public class Main {

public static void

main(String[] args) {       String fileName
= "test.txt";

      try {
         //        WRITE    operation    (overwrite)
FileWriter   fw   =   new   FileWriter(fileName);
fw.write("Hello World\n");              fw.write("This
is write operation\n");             fw.close();

         System.out.println("Write operation completed.");

         //APPEND operation
         FileWriter  fw2  =  new  FileWriter(fileName,  true);
fw2.write("This is append operation\n");            fw2.close();

         System.out.println("Append operation completed.");

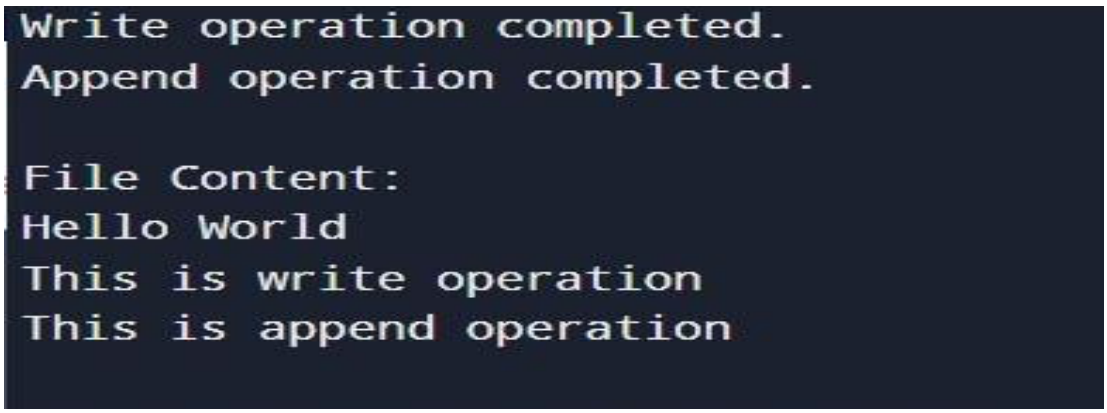
         //READ operation
         FileReader fr = new FileReader(fileName);
         BufferedReader br = new BufferedReader(fr);

         String line;
         System.out.println("\nFile Content:");          while
((line = br.readLine()) != null) {
            System.out.println(line);
         }

br.close();

      } catch (IOException e) {
         System.out.println("Error: " + e);
      }
   }
}
```

OUTPUT :


Write operation completed.
Append operation completed.
File Content:
Hello World
This is write operation
This is append operation

PRACTICAL – 09

AIM :

Create ArrayList mover. In which selected element of one list moved to another. i.e.
Subject Selected Subject Java
DAA
Maths-I EME
Database
Lets move Java and DAA to selected subject Subject
Selected Subject
Maths-I Java
EME DAA
Database

CODE :

```
import java.util.ArrayList;

public class Main {
   public static void main(String[] args) {

       // Subject list
       ArrayList<String> subject = new ArrayList<>();
subject.add("Java");                    subject.add("DAA");
subject.add("Maths-I");                 subject.add("EME");
subject.add("Database");

       // Selected Subject list
       ArrayList<String> selected = new ArrayList<>();
       // Print initial lists
       System.out.println("Before Moving:\n");

       System.out.println("Subject:");      for
(String      s      :      subject)      {
System.out.println(s);
       }
       System.out.println("\nSelected     Subject:");
for      (String      s      :      selected)      {
System.out.println(s);
       }
    //      Move      Java      and      DAA
selected.add("Java");      selected.add("DAA");
subject.remove("Java");
subject.remove("DAA");
System.out.println("\nAfter Moving:\n");
       System.out.println("Subject:");      for
(String      s      :      subject)      {
System.out.println(s);
```

```
        }

        System.out.println("\nSelected Subject:");
for (String s : selected) {
System.out.println(s);
        }
    }
}
```

OUTPUT :

```
Before Moving:

Subject:
Java
DAA
Maths-I
EME
Database

Selected Subject:

After Moving:

Subject:
Maths-I
EME
Database

Selected Subject:
Java
DAA
```

PRACTICAL -09

AIM :

Create a generic class Box<T> with a method addItem (T item) that stores items in an ArrayList<T>. Demonstrate the class by:

- Creating a Box<String> for names.
- Creating a Box<Integer> for roll numbers.
- Display stored items for both.

CODE :

```
import java.util.ArrayList;

// Generic class class
Box<T> {
   ArrayList<T> items = new ArrayList<>();

   // Method to add item     void
addItem(T item) {
      items.add(item);
   }

   // Method to display items     void
display() {        for (T item :
items) {
System.out.println(item);
      }
   }
}

// Main class public class Main {
public static void main(String[] args) {

      // Box for names
      Box<String> nameBox = new Box<>();
nameBox.addItem("Rahul");
nameBox.addItem("Priya");
nameBox.addItem("Amit");

      // Box for roll numbers
      Box<Integer> rollBox = new Box<>();
rollBox.addItem(101);
rollBox.addItem(102);
rollBox.addItem(103);

      // Display items
      System.out.println("Names:");
nameBox.display();
```

```
        System.out.println("\nRoll Numbers:");
        rollBox.display();
    }
}
```

OUTPUT :



PRACTICAL -10

AIM :

Write a javafx form for user signup which include esusername(Textfield), password(passwordfield), gender(radiobutton), branch(combobox), address(textarea),hobbies(checkbox). Create two buttons display valueof user data into a label. signup(button), clear(button).

CODE :

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;
public class Main extends Application {
public void start(Stage stage) {

    // Controls
    TextField username = new TextField();
    PasswordField password = new PasswordField();

    RadioButton male = new RadioButton("Male");
    RadioButton female = new RadioButton("Female");
ToggleGroup genderGroup = new ToggleGroup();
male.setToggleGroup(genderGroup);
female.setToggleGroup(genderGroup);

    ComboBox<String> branch = new
ComboBox<>();       branch.getItems().addAll("CE",
"IT", "ME", "EC");      TextArea address = new
TextArea();

    CheckBox reading = new CheckBox("Reading");
CheckBox sports = new CheckBox("Sports");

    Button signup = new Button("Signup");
    Button clear = new Button("Clear");

    Label result = new Label();

    // Layout
    GridPane grid = new GridPane();
grid.setPadding(new Insets(10));
grid.setHgap(10);       grid.setVgap(10);
```

```
        // Add components        grid.add(new
Label("Username:"), 0, 0);
grid.add(username, 1, 0);

        grid.add(new Label("Password:"), 0, 1);
grid.add(password, 1, 1);

        grid.add(new Label("Gender:"), 0,
2);        grid.add(male, 1, 2);
grid.add(female, 2, 2);

        grid.add(new Label("Branch:"), 0, 3);
grid.add(branch, 1, 3);

        grid.add(new Label("Address:"), 0, 4);
grid.add(address, 1, 4);

        grid.add(new Label("Hobbies:"), 0,
5);        grid.add(reading, 1, 5);
grid.add(sports, 2, 5);
grid.add(signup, 0, 6);
grid.add(clear, 1, 6);
grid.add(result, 0, 7, 3, 1);

        // Signup button action
signup.setOnAction(e -> {
String user = username.getText();

            String pass = password.getText();

            String gender = "";            if
(male.isSelected()) gender = "Male";            if
(female.isSelected()) gender = "Female";

            String br = branch.getValue();

            String addr = address.getText();

            String hobbies = "";            if
(reading.isSelected()) hobbies += "Reading ";            if
(sports.isSelected()) hobbies += "Sports ";

            result.setText("Username: " + user +
                    "\nPassword: " + pass +
                    "\nGender: " + gender +
                    "\nBranch: " + br +
                    "\nAddress: " + addr +
                    "\nHobbies: " + hobbies);
        });
```

```
        // Clear button action
clear.setOnAction(e -> {
username.clear();
password.clear();
genderGroup.selectToggle(null);
branch.setValue(null);
address.clear();
reading.setSelected(false);
sports.setSelected(false);
result.setText("");
        });

        // Scene
        Scene scene = new Scene(grid, 400, 400);
stage.setTitle("Signup Form");
stage.setScene(scene);
stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:

```
Enter Username: ABC
Enter Password: 1234
Enter Gender (Male/Female): MALE
Enter Branch (CE/IT/ME/EC): CE
Enter Address: C-BULDING
Enter Hobbies (Reading/Sports): READING

--- User Details ---
Username: ABC
Password: 1234
Gender: MALE
Branch: CE
Address: C-BULDING
Hobbies: READING

Form Cleared!
```

PRACTICAL – 10

AIM :

Write a javafx program to move a circle from left to right when navigation keys been clicked. when mouse hovered over the circle then it change the color of circle.

CODE :

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.input.KeyCode;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.stage.Stage;
public class Main extends Application {
public void start(Stage stage) {

    Pane root = new Pane();

    // Create circle
    Circle circle = new Circle(50, 100, 30);
circle.setFill(Color.BLUE);
root.getChildren().add(circle);

    Scene scene = new Scene(root, 400, 300);
    // Move circle using arrow keys
scene.setOnKeyPressed(e -> { if
(e.getCode() == KeyCode.RIGHT) {
        circle.setCenterX(circle.getCenterX() + 10);
} else if (e.getCode() == KeyCode.LEFT) {
        circle.setCenterX(circle.getCenterX() - 10);
      }
});

    // Change color on mouse hover
circle.setOnMouseEntered(e -> {
      circle.setFill(Color.RED);
    });

    circle.setOnMouseExited(e -> {
      circle.setFill(Color.BLUE);
    });
    stage.setTitle("Move Circle");
stage.setScene(scene); stage.show();

    // Important: request focus for key events
```

```
scene.requestFocus();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

OUTPUT :

```
Circle Position: 0 | Color: BLUE
Press:
1 - Move Right
2 - Move Left
3 - Hover (Change Color)
4 - Exit
1

Circle Position: 10 | Color: BLUE
Press:
1 - Move Right
2 - Move Left
3 - Hover (Change Color)
4 - Exit
2
```

```
Circle Position: 0 | Color: BLUE
Press:
1 - Move Right
2 - Move Left
3 - Hover (Change Color)
4 - Exit
3

Circle Position: 0 | Color: RED
Press:
1 - Move Right
2 - Move Left
3 - Hover (Change Color)
4 - Exit
4
Program Ended
```